Practical Machine Learning - Comprehensive Cheatsheet

CCEE (CDAC) Exam Preparation

Topics Covered: Sessions 2–8 (Bias-Variance, VC Dimension, SRM, Model Selection & Validation, Scikit-Learn, Clustering, PCA, Metrics, Decision Trees, Ensemble Methods)

SESSION 1: INFORMATION THEORY & INTRODUCTION TO ML

(Quick recap from earlier sessions)

Information Theory Basics

- **Self-information**: $I(x) = \log_2(1/P(x))$; higher for rare events
- **Entropy**: $H(X) = -\Sigma P(x) \cdot \log_2(P(x))$; measures average uncertainty
- **Mutual information**: $I(X;Y)$; shared information between variables
- **Kullback-Leibler divergence**: Measure of how one distribution differs from another

Machine Learning Types

- **Supervised**: Regression, Classification (labels provided)
- **Unsupervised**: Clustering, Dimensionality reduction (no labels)
- **Semi-supervised**: Mix of labeled and unlabeled data
- **Reinforcement**: Learn through rewards and penalties

SESSION 2: BIAS-COMPLEXITY TRADE-OFF, VC DIMENSION, SRM, REGULARIZATION

1. Bias–Complexity (Bias–Variance) Trade-off

Bias: Systematic error from overly restrictive model assumptions (underfitting).

- Signs: High training error + high test error; model systematically wrong.

Variance: Error from model being too sensitive to training data fluctuations (overfitting).

- Signs: Very low training error but high test error.

Trade-off: As model complexity ↑ → bias ↓ but variance ↑

Key formula:
Generalization Error ≈ Bias^2 + Variance + Irreducible Error

MCQ tips:

- High bias → underfitting
- High variance → overfitting
- Need balance between underfitting and overfitting

---

2. VC Dimension

Definition: Largest set of points that can be **shattered** by a hypothesis class H.

Shattering: A hypothesis class H shatters a set S if for every possible binary labeling of S, there exists some classifier h ∈ H that realizes that labeling.

Examples:

- Linear separators in 2D: VC dim = 3 (can shatter any 3 non-collinear points)
- Linear separators in d-dimensions: VC dim = d+1

Key facts:

- VC dimension measures **capacity** of hypothesis class
- Higher VC dim → larger capacity → higher risk of overfitting if sample small
- Infinite VC dimension = can represent arbitrarily complex functions

MCQ tips:

- "VC dimension measures capacity/complexity" → TRUE
- "VC dimension = number of parameters" → FALSE in general
- VC dim is about largest number of points that CAN be shattered

---

3. Non-Uniform Learnability

a) Structural Risk Minimization (SRM)

Idea: Organize hypotheses into nested structures $H_1 \subset H_2 \subset ...$ with increasing complexity.

Objective:
Choose h that minimizes: Empirical Risk + Complexity Penalty

Penalty typically based on VC dimension or parameter norm (L1, L2).

Example implementations: Ridge regression, Lasso, SVM with regularization.

Key insight: Trades off training error vs model complexity explicitly.

b) Occam's Razor

Principle: Among hypotheses fitting data equally well, prefer the **simplest** one.

Justification: Simpler classes (lower VC dim, shorter description length) have better generalization bounds.

MCQ tips:

- "Prefer simplest hypothesis that explains data" → TRUE

- "Always choose lowest training error" → FALSE (that's ERM, can overfit)

c) No Free Lunch (NFL) Theorem

Statement: Averaged over ALL possible data distributions, no learning algorithm is universally better than any other.

Implication: To do better than random, an algorithm must embed assumptions about the problem (inductive bias).

Why it matters:

- No single "best" classifier for all tasks
- Domain knowledge and assumptions are crucial
- Different algorithms excel in different problem spaces

MCQ tips:

- "One algorithm best for all problems" → FALSE
- "NFL motivates need for inductive bias" → TRUE

---

4. Regularization and Stability

Regularization: Adding explicit penalty to loss function to discourage overly complex models.

Typical objective:
Total Loss = Empirical Loss + $\lambda \times$ Regularizer

Common regularizers:

Type	Formula	Effect
L2 (Ridge)	$\lambda \|w\|_2^2$	Shrinks weights smoothly; keeps all features
L1 (Lasso)	$\lambda \|w\|_1$	Encourages sparsity; exactly zeros some weights

Effects of regularization:

- Controls effective model capacity even if hypothesis class is rich
- Reduces variance, may increase bias slightly → helps reach good balance
- Improves generalization by preventing overfitting

Algorithmic Stability: A learning algorithm is stable if small changes in training data (remove/replace one sample) don't significantly change the learned model or predictions.

Connection:

- Regularized algorithms tend to be more stable
- More stability → better generalization

- Stability is an alternative theoretical guarantee for generalization

MCQ tips:

- "Regularization added to loss" → TRUE
- "L2 encourages large weights" → FALSE (encourages smaller)
- "Stability = insensitivity to training data changes" → TRUE

SESSION 3: MODEL SELECTION & VALIDATION, SCIKIT-LEARN

1. Train–Validation–Test Split

Train set: Learn model parameters.
Validation set: Compare models/hyperparameters; pick best.
Test set: Final unbiased performance estimate; never touch during model selection.

Typical split: 60% train, 20% validation, 20% test (or 80% train, 20% test if no explicit validation).

Why needed: Training error is optimistic; need separate validation/test to estimate true generalization.

---

2. Cross-Validation (CV)

k-fold CV:

1. Split data into k folds
2. For each fold i: train on k−1 folds, validate on fold i
3. Average performance across k folds

Advantages:

- More robust than single train–test split
- Uses data efficiently (each sample used for training k−1 times, validation once)
- Better for model selection and hyperparameter tuning

Each sample: Used for training k−1 times, validation exactly 1 time.

MCQ tips:

- "Validation set = final unbiased estimate" → FALSE (that's test set)
- "Validation set = for model selection" → TRUE
- "k-fold CV: each sample validated exactly once" → TRUE

---

3. Scikit-Learn Estimator API

Core philosophy: Every model/preprocessor follows a consistent interface.

Key methods:

- `fit(X, y)`: Learn from data
- `predict(X_new)`: Make predictions

- transform(X): Transform features (preprocessors)
- fit_transform(X): Fit and transform in one step (faster)

Data representation:

- X: 2D array-like (samples × features)
- y: 1D array (target vector)

Hyperparameters vs Parameters:

- **Hyperparameters**: Set when creating estimator (e.g., n_neighbors=5)
- **Parameters**: Learned by fit(); accessible with trailing underscore (e.g., coef_, intercept_)

Standard workflow:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

1. Split data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

2. Preprocess (fit on train, apply to train & test)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3. Train model

```
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_scaled, y_train)
```

4. Evaluate

```
y_pred = clf.predict(X_test_scaled)
```

Pipeline: Chains preprocessing + model into one object; ensures same transformations applied consistently.

MCQ tips:

- "Estimator trained via train() method" → FALSE; it's fit()
- "fit(X, y) then predict(X_new)" → TRUE typical pattern
- "Hyperparameters set at instantiation" → TRUE
- "Parameters end with _" → TRUE

SESSION 4 & 5: CLUSTERING, PCA, KERNEL PCA, RANDOM PROJECTIONS

1. Clustering Basics

Unsupervised learning: partition data into clusters such that points in same cluster are more similar than points in different clusters.

Input: Only features X; no labels y.
Output: Cluster assignments (e.g., cluster IDs 1, 2, 3, ...).

Common types:

- Partitioning: K-means, K-medoids
- Hierarchical: agglomerative, divisive
- Density-based: DBSCAN

---

2. K-Means Clustering

Goal: Minimize within-cluster sum of squares (WCSS).

Algorithm:

1. Initialize K centroids
2. Assignment: Assign each point to nearest centroid (typically Euclidean distance)
3. Update: Recompute each centroid as mean of its assigned points
4. Repeat until convergence

Characteristics:

- Must specify K in advance
- Works best for roughly spherical, similar-sized clusters
- Sensitive to initialization and outliers
- Fast; scales to large data

Limitations:

- Requires pre-specifying K
- May get stuck in local optima
- Sensitive to initialization; use k-means++ for better initialization

---

3. Hierarchical Clustering

Agglomerative (bottom-up):

- Start: each point is its own cluster
- Iteratively: merge two closest clusters until one cluster remains
- Output: **dendrogram** (tree structure)

Divisive (top-down): Start with one cluster, recursively split.

Linkage criteria (for distance between clusters):

- **Single linkage**: Distance between closest pair → tends to create chain-like clusters
- **Complete linkage**: Distance between farthest pair → compact clusters
- **Average linkage**: Average pairwise distance
- **Ward's linkage**: Merges clusters minimizing increase in within-cluster variance

Advantages:

- No need to pre-specify K; choose by cutting dendrogram
- Shows hierarchical relationship between clusters
- More interpretable with dendrograms

Disadvantage: Computationally expensive for large datasets.

MCQ tips:

- "Single linkage = closest pair distance" → TRUE
- "Ward's linkage = minimize SSE increase" → TRUE
- "Hierarchical produces dendrogram" → TRUE

---

4. Distance Measures & Data Preparation

Common distances:

- **Euclidean**: Straight-line distance; used in K-means
- **Manhattan**: Sum of absolute differences
- **Cosine**: 1 − cosine similarity; ignores magnitude, only direction

Scaling importance:

- If one feature has much larger numeric range than another, it dominates Euclidean distance
- Standard practice: scale/standardize features before K-means (zero mean, unit variance)
- Use `StandardScaler` in scikit-learn

Weighting: Can assign weights to features to reflect importance in distance computation.

---

5. Cluster Evaluation & Profiling

Internal measures (no true labels):

Silhouette coefficient: For point i:
a(i) = mean distance to points in own cluster (cohesion)
b(i) = smallest mean distance to points in any other cluster (separation)

Silhouette(i) = [b(i) − a(i)] / max(a(i), b(i))

Range: −1 to +1
≈ +1: well-clustered
≈ 0: on boundary
< 0: possibly wrong cluster

Overall silhouette = average over all points; higher is better.

External measures (need true labels):

- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)

Cluster profiling: Examine feature statistics (mean, median, distribution) in each cluster to interpret what each cluster represents.

---

6. PCA (Principal Component Analysis)

Goal: Find new orthogonal axes (principal components) such that:

1. First component has maximum variance
2. Second has next highest variance, etc.
3. Keep only first k components to reduce dimensionality

Key math:

- Components = **eigenvectors of covariance matrix**
- Importance = **eigenvalues** (proportional to explained variance)
- Components are orthogonal and ordered by explained variance

Steps:

1. Standardize data (zero mean, often unit variance)
2. Compute covariance matrix
3. Compute eigenvectors & eigenvalues
4. Sort eigenvectors by descending eigenvalues; pick top k
5. Project data onto these k eigenvectors

Explained variance ratio:
Variance explained by component i = eigenvalue(i) / sum(all eigenvalues)
Choose k such that cumulative explained variance ≥ 90–95%.

Properties:

- **Linear** dimensionality reduction
- **Unsupervised** (doesn't use labels)
- Interpretable: loadings show how original features contribute
- First PC = direction of maximum variance

MCQ tips:

- "First PC = direction of maximum variance" → TRUE
- "Components are eigenvectors of covariance" → TRUE
- "PCA is linear" → TRUE
- "Explained variance = eigenvalue / sum(eigenvalues)" → TRUE

---

7. Kernel PCA

Problem: Standard PCA is linear; cannot capture nonlinear structures (e.g., spirals).

Solution: Apply PCA in implicit high-dimensional feature space using kernel trick.

Key idea:

- Map data via kernel function: $K(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j) \rangle$
- Perform PCA using kernel (Gram) matrix instead of covariance matrix
- Can capture nonlinear manifolds

Common kernels: RBF (Gaussian), polynomial, sigmoid.

Advantages: Captures nonlinear structure.
Disadvantages: More expensive ($n \times n$ kernel matrix); less interpretable.

MCQ tips:

- "Kernel PCA = nonlinear PCA via kernel trick" → TRUE
- "Uses kernel/Gram matrix, not covariance" → TRUE
- "Good for nonlinear manifolds" → TRUE

---

8. Random Projections

Motivation: For very high-dimensional data, PCA/Kernel PCA can be expensive. Random projections offer a cheap alternative.

Johnson–Lindenstrauss Lemma: Points in high-dimensional space can be projected into lower-dimensional space (dimension $\propto \log n$) such that pairwise distances are preserved up to small distortion $(1 \pm \varepsilon)$.

Method:

- Create random matrix R (Gaussian or sparse random entries)
- Scale appropriately
- New data: $X' = X \times R \rightarrow$ k-dimensional representation

Advantages:

- Very fast; no covariance/eigenvalues needed
- Good for large-scale, high-dimensional problems
- Distance-preserving probabilistically

Disadvantage: No interpretability like PCA components.

MCQ tips:

- "Based on Johnson–Lindenstrauss lemma" → TRUE
- "Very fast for large-scale problems" → TRUE
- "Preserves distances probabilistically" → TRUE

Summary: Dimensionality Reduction

Aspect	PCA	Kernel PCA	Random Projections
Nature	Linear	Nonlinear (via kernel)	Approximate linear
Speed	Moderate	Slow (n×n matrix)	Very fast
Interpretability	High (loadings)	Low	None
Use case	Linear structure, visualization	Nonlinear manifolds	Large-scale high-dim

---

SESSION 6: EVALUATION METRICS (CLASSIFICATION & REGRESSION)

1. Classification Metrics

Confusion Matrix

```
                Predicted
              Positive  Negative
```

Actual Pos. TP FN
Neg. FP TN

Accuracy

Accuracy = (TP + TN) / (TP + TN + FP + FN)

- Fraction of correct predictions
- **Misleading on imbalanced data** (e.g., 99% zeros → 99% accuracy by always predicting zero)

Precision & Recall

Precision = TP / (TP + FP) [Of predicted positives, how many correct?]
Recall = TP / (TP + FN) [Of actual positives, how many caught?]

F1-Score

$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

- Harmonic mean of precision and recall
- Useful for **imbalanced data** or when both FP and FN matter
- Punishes if either precision or recall is low

ROC & AUC

- **ROC curve**: Plots TPR (y-axis) vs FPR (x-axis) for varying classification thresholds
- **AUC** (Area Under Curve): Single number 0 to 1; measures ranking quality
- AUC ≈ probability classifier ranks random positive higher than random negative
- Uses **predicted scores/probabilities**, not just class labels

Log Loss (Cross-Entropy)

$\text{LogLoss} = -[y \times \log(p) + (1-y) \times \log(1-p)]$

- Penalizes wrong AND overconfident predictions heavily
- Lower is better
- Use when evaluating **quality of predicted probabilities**

MCQ tips:

- Use **Accuracy** for balanced data
- Use **F1** for imbalanced data
- Use **ROC AUC** for ranking quality across thresholds
- Use **Log Loss** for probability calibration

---

2. Regression Metrics

MAE (Mean Absolute Error)

$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$

- Average absolute error
- Robust to outliers
- Same units as target

RMSE (Root Mean Squared Error)

$MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$
$RMSE = \sqrt{MSE}$

- Penalizes larger errors more heavily (squaring)
- Same units as target
- Use when large errors especially bad

R^2 Score (Coefficient of Determination)

$R^2 = 1 - (SS_{res} / SS_{tot})$

$SS_{res} = \Sigma(y_i - \hat{y}_i)^2$ [residual sum of squares]
$SS_{tot} = \Sigma(y_i - \bar{y})^2$ [total sum of squares]

- Measures proportion of variance in y explained by model
- $\mathbf{R^2 = 1}$: Perfect fit
- $\mathbf{R^2 = 0}$: Predicts no better than mean
- $\mathbf{R^2 < 0}$: Worse than predicting mean

MCQ tips:

- MAE, RMSE, R^2 are **regression** metrics
- F1, Accuracy, ROC AUC, Log Loss are **classification** metrics

---

SESSION 7: DECISION TREES & CART

1. Decision Trees

A tree-structured model with decision rules at internal nodes and predictions at leaves.

Structure:

- **Root node**: First split
- **Internal nodes**: Decision tests on features
- **Leaf nodes**: Predictions (class label or numeric value)

Advantages:

- Highly **interpretable**: explicit if-then rules from root to leaf
- Handles both numeric and categorical features
- Can model nonlinear boundaries

Disadvantages:

- Prone to **overfitting** if grown deep
- Single tree unstable (small data changes → big tree changes)
- Often less accurate than ensembles

---

2. Splitting Criteria

Entropy (Information Theory)

$H = -\Sigma\, p_j \times \log_2(p_j)$

- Measures disorder; 0 for pure node (all one class)
- Higher when classes mixed

Information Gain

$IG = H(\text{parent}) - \Sigma\, w_i \times H(\text{child}_i)$

- Reduction in entropy after split
- Choose split with **highest IG**

Gini Impurity

$G = 1 - \Sigma\, p_j^2$

- Also 0 for pure node
- Maximum when classes evenly mixed
- Slightly faster than entropy (no logs)

Key insight: Both entropy and Gini measure node **impurity**; lower is better.

MCQ tips:

- "Gini formula = $1 - \Sigma\, p_j^2$" → TRUE
- "Entropy formula = $-\Sigma\, p_j \times \log_2(p_j)$" → TRUE
- Both measure **impurity**; lower is purer

---

3. CART (Classification and Regression Trees)

Framework for building both:

- **Classification trees**: Target categorical
- **Regression trees**: Target continuous

Key features:

- Uses **binary splits only**: each node splits into exactly 2 children
- **Recursive binary splitting**: choose feature & threshold minimizing impurity (classification) or error (regression)

For classification:

- Impurity measure: **Gini** (or entropy)
- Leaf prediction: Majority class

For regression:

- Impurity measure: **Sum of Squared Errors (SSE)** or reduction in SSE (RSS)
- Leaf prediction: Mean of target values in leaf

Pruning:

- Fully grown trees **overfit**
- **Cost-complexity pruning**: balance tree size vs training error
- Cuts back branches that add little performance gain

MCQ tips:

- "CART uses binary splits" → TRUE
- "CART for both classification and regression" → TRUE

- "Classification leaf = majority class" → TRUE
- "Regression leaf = mean of targets" → TRUE
- "Pruning reduces overfitting" → TRUE

SESSION 8: ENSEMBLE METHODS

1. Random Forest

Idea: Many decision trees + averaging/voting

Key mechanics:

- **Bagging**: Each tree trained on bootstrap sample (random rows with replacement)
- **Feature sampling**: At each split, consider random subset of features, not all
- **Combination**:
  - Classification: majority vote
  - Regression: average predictions

Why it works:

- Individual trees are high-variance; averaging reduces variance
- Row + feature sampling make trees diverse, so averaging is effective

Characteristics:

- Mainly **reduces variance**
- Handles high-dimensional data
- Estimates feature importance
- Out-of-bag (OOB) samples can estimate test error without separate validation

MCQ tips:

- "RF reduces variance" → TRUE
- "Bagging = bootstrap sampling" → TRUE
- "Combines trees via vote (classify) or average (regress)" → TRUE

---

2. Gradient Boosting Machines (GBM/GBDT)

Idea: Build trees **sequentially**, each correcting errors of the current model.

High-level algorithm:

1. Start with simple model $F_0(x)$ (e.g., constant)
2. At step m:
   - Compute residuals or negative loss gradients
   - Fit decision tree to these residuals
3. Update: F_{m+1}(x) = F_m(x) + η × f_m(x), where η = learning rate
4. Repeat for M iterations

Key points:

- Trees are typically **shallow** (weak learners)

- **Learning rate η** controls each tree's contribution; smaller → more trees needed but often better generalization
- Works with any differentiable loss (log loss, squared error, etc.)

Bagging vs Boosting:

Aspect	Bagging / RF	Gradient Boosting
Training	Independent, parallel	Sequential
Goal	Reduce variance	Reduce bias, fit patterns
Combination	Average/vote	Weighted sum (additive)

MCQ tips:

- "GBM builds trees sequentially" → TRUE
- "Each tree fits residuals of previous" → TRUE
- "Learning rate controls tree contribution" → TRUE
- "GBM mainly reduces bias" → TRUE

---

3. XGBoost, LightGBM, CatBoost

All are **gradient boosted decision tree** libraries, optimized for speed and accuracy.

XGBoost

- "Extreme Gradient Boosting"
- Uses **second-order gradients** (Hessian) for better optimization
- Built-in **L1/L2 regularization**
- Handles missing values by learning default splits
- Popular in competitions

LightGBM

- Designed for **speed & large datasets**
- **Leaf-wise growth**: splits leaf with largest loss reduction (deeper, more accurate trees but risk overfitting)
- **Histogram-based binning**: discretizes features to speed up splits & reduce memory
- Good for millions of rows, high-dimensional data

CatBoost

- Optimized for **categorical features**
- "Ordered boosting" and target statistics to reduce target leakage
- Works well out-of-the-box with minimal hyperparameter tuning
- Strong handling of categorical features

MCQ tips:

- "All are gradient boosting variants" → TRUE
- "LightGBM best for large-scale data" → TRUE
- "XGBoost best for fine-tuning & competitions" → TRUE

- "CatBoost best with categorical features" → TRUE

---

4. Model Stacking

Idea: Combine different base models via a meta-model.

Levels:

- **Level-0**: Base learners (e.g., RF, GBM, logistic regression, SVM)
- **Level-1**: Meta-model that takes base model outputs as input features

Goal: Exploit complementary strengths of different algorithms.

MCQ tips:

- "Stacking = meta-model on base model predictions" → TRUE
- "Combines different algorithm types" → TRUE

QUICK MCQ REFERENCE BY TOPIC

Bias-Variance

- High bias = underfitting
- High variance = overfitting
- Trade-off: ↑ complexity → ↓ bias, ↑ variance

VC Dimension

- Measures hypothesis class capacity
- = Largest set that can be shattered
- Linear classifiers in 2D: VC dim = 3

Regularization

- L2 (Ridge): Shrinks weights smoothly
- L1 (Lasso): Encourages sparsity
- Both reduce overfitting

Validation

- Train set: Learn parameters
- Validation set: Choose models/hyperparameters
- Test set: Final unbiased estimate (never use for selection)

Cross-Validation

- k-fold: Each sample used for training $k-1$ times, validation 1 time
- More robust than single split

Scikit-Learn

- Estimators: `fit()`, `predict()`, `transform()`
- Hyperparameters: Set at instantiation
- Parameters: Learned; end with `_`
- Pipeline: Chains preprocessing + model

Clustering

- **K-means**: Minimize WCSS; specify K
- **Hierarchical**: Produces dendrogram; choose K from tree
- **Silhouette**: Range −1 to +1; higher better

PCA

- Linear dimensionality reduction
- Components = eigenvectors of covariance
- First PC = direction of max variance
- Explained variance = eigenvalue / sum(eigenvalues)

Kernel PCA

- Nonlinear extension via kernel trick
- Good for nonlinear manifolds
- Uses kernel (Gram) matrix

Metrics (Classification)

- **Accuracy**: Overall correctness (bad for imbalance)
- **F1**: Harmonic mean precision/recall (good for imbalance)
- **ROC AUC**: Ranking quality across thresholds (uses scores/probabilities)
- **Log Loss**: Probability quality; penalizes overconfidence

Metrics (Regression)

- **MAE**: Average absolute error; robust to outliers
- **RMSE**: Penalizes large errors more
- $\mathbf{R^2}$: Proportion of variance explained (0 to 1)

Decision Trees

- Splitting: Gini impurity or entropy
- Gini = $1 - \Sigma\ p_j^2$
- Entropy = $-\Sigma\ p_j \times \log_2(p_j)$
- Pruning reduces overfitting

CART

- Binary splits (2 children per node)
- Classification: Gini → majority class at leaf
- Regression: RSS → mean of targets at leaf

Random Forest

- Bagging + feature sampling
- Reduces variance
- Combines via vote (classify) or average (regress)

Gradient Boosting

- Sequential tree building
- Each tree fits residuals
- Reduces bias
- Learning rate controls contribution

XGBoost / LightGBM / CatBoost

- All gradient boosting variants
- XGBoost: 2nd-order gradients, regularization
- LightGBM: Leaf-wise growth, histograms, speed
- CatBoost: Categorical features, minimal tuning

Model Stacking

- Meta-model combines base model predictions
- Exploits complementary strengths

FORMULA SHEET (Quick Reference)

Concept	Formula
Accuracy	(TP+TN)/(TP+TN+FP+FN)
Precision	TP/(TP+FP)
Recall (TPR)	TP/(TP+FN)
F1	2×(Prec×Rec)/(Prec+Rec)
FPR	FP/(FP+TN)
MAE	(1/n)Σ|y_i−ŷ_i|
RMSE	√[(1/n)Σ(y_i−ŷ_i)²]
R^2	1 − SS_res/SS_tot
Gini	1 − Σp_j²
Entropy	−Σp_j×log₂(p_j)
Info Gain	H(parent) − Σw_i×H(child_i)
LogLoss	−[y×log(p)+(1−y)×log(1−p)]
Silhouette	[b(i)−a(i)]/max(a(i),b(i))
Explained Variance Ratio	eigenvalue_i / Σ(all eigenvalues)

EXAM-FOCUSED TIPS

1. **Always choose the right metric**:
   - Imbalanced data → F1, not Accuracy
   - Probability quality → Log Loss
   - Regression → MAE, RMSE, R^2
2. **Bias-Variance**:
   - ↑ complexity → ↓ bias, ↑ variance
   - Sweet spot minimizes total error
3. **Validation matters**:
   - Train/val/test split is crucial
   - Never use test set for model selection
4. **Ensemble** > **Single Tree**:
   - RF reduces variance via bagging
   - GBM reduces bias via sequential correction
   - Combine different algorithms via stacking
5. **Dimensionality reduction**:
   - Linear: PCA (fast, interpretable)
   - Nonlinear: Kernel PCA
   - Large-scale: Random projections

6. **Tree splitting**:
    - Gini = $1 - \Sigma p_j^2$ (CART default)
    - Entropy = $-\Sigma p_j \times \log_2(p_j)$
    - Both measure impurity; lower is better
7. **Scikit-Learn workflow**:
    - `train_test_split()` → `fit()` → `predict()`
    - Use `Pipeline` to avoid data leakage
    - Always scale features for distance-based algorithms
8. **Clustering evaluation**:
    - Silhouette ≈ +1 → excellent clusters
    - Silhouette ≈ 0 → on boundary
    - Silhouette < 0 → possibly wrong cluster
9. **Model selection**:
    - Use validation set to compare hyperparameters
    - Use test set only for final evaluation
    - Cross-validation more robust than single split
10. **Ensemble libraries**:
    - XGBoost → fine-tuning & competitions
    - LightGBM → large-scale, high-dimensional
    - CatBoost → categorical features, minimal tuning

KEY LEARNING OUTCOMES

By this point, you understand:

✓ **Theory**: Bias-variance trade-off, VC dimension, SRM, regularization, stability
✓ **Practical ML**: Model selection, validation, scikit-learn API, preprocessing
✓ **Unsupervised Learning**: K-means, hierarchical clustering, PCA, Kernel PCA, random projections
✓ **Evaluation**: Classification metrics (accuracy, F1, ROC AUC, log loss), regression metrics (MAE, RMSE, R^2)
✓ **Trees & Ensembles**: Decision trees, CART, random forests, gradient boosting, XGBoost, stacking

---

Last Updated: January 9, 2026
Coverage: Sessions 2–8 (Practical Machine Learning - Bias-Variance through Ensemble Methods)
Exam Focus: CCEE (CDAC) with MCQ-style questions and formulas